TP n° 5 : Transactions et niveaux d'isolation

Exercice 1 :

Number	Elapsed	Statement	Feedback	Rows
1	0.11	BEGIN EXECUTE IMMEDIATE 'DROP TABLE transaction_tp PURG	Statement processed.	1
2	0.04	CREATE TABLE transaction_tp (idTransaction VARCHAR2(44	Table created.	0
3	0.02	INSERT INTO transaction_tp VALUES ('T1', 100)	1 row(s) inserted.	1
4	0.00	INSERT INTO transaction_tp VALUES ('T2', 200)	1 row(s) inserted.	1
5	0.01	INSERT INTO transaction_tp VALUES ('T3', 300)	1 row(s) inserted.	1
6	0.02	UPDATE transaction_tp SET valTransaction = 250 WHERE idTra	1 row(s) updated.	1
7	0.01	DELETE FROM transaction_tp WHERE idTransaction = 'T3'	1 row(s) deleted.	1
8	0.01	SELECT * FROM transaction_tp	2 rows selected.	2
9	0.00	ROLLBACK	Statement processed.	0
10	0.00	SELECT * FROM transaction_tp	Statement processed.	0
11	0.00	INSERT INTO transaction_tp VALUES ('T10', 1000)	1 row(s) inserted.	1
12	0.00	INSERT INTO transaction_tp VALUES ('T11', 1100)	1 row(s) inserted.	1
13	0.01	SELECT * FROM transaction_tp	2 rows selected.	2
14	0.00	SELECT * FROM transaction_tp	2 rows selected.	2
15	0.01	INSERT INTO transaction_tp VALUES ('T20', 2000)	1 row(s) inserted.	1

Download

row(s) 1 - 15 of 23 Next ▸

23	23	0
Statements Processed	Successful	With Errors

🔹 1. Création de la table

```
CREATE TABLE transaction_tp (
  idTransaction VARCHAR2(44),
  valTransaction NUMBER(10)
);
```

Cette table permet de tester les différentes opérations liées aux transactions.

🔹 2. Test du ROLLBACK

Manipulations (Session S2)

```
INSERT INTO transaction_tp VALUES ('T1', 100);
INSERT INTO transaction_tp VALUES ('T2', 200);
INSERT INTO transaction_tp VALUES ('T3', 300);

UPDATE transaction_tp
SET valTransaction = 250
WHERE idTransaction = 'T2';

DELETE FROM transaction_tp
```

WHERE idTransaction = 'T3';

SELECT * FROM transaction_tp;

ROLLBACK;

SELECT * FROM transaction_tp;

✔ **Observation**

- Avant `ROLLBACK` :
  Les modifications sont visibles (T1, T2 modifié, T3 supprimé)
- Après `ROLLBACK` :
  La table revient à son état initial

✔ **Conclusion**

Le `ROLLBACK` annule toutes les modifications non validées de la transaction.

🔹 3. Fermeture sans COMMIT

Manipulations (Session S2)

INSERT INTO transaction_tp VALUES ('T10', 1000);
INSERT INTO transaction_tp VALUES ('T11', 1100);

SELECT * FROM transaction_tp;

Puis fermer la session sans exécuter `COMMIT`.

Vérification (Session S1)

SELECT * FROM transaction_tp;

✔ **Observation**

Les données insérées ne sont pas visibles dans une autre session.

✔ **Conclusion**

Fermer une session sans `COMMIT` entraîne un **ROLLBACK automatique**.

🔹 4. Fermeture brutale

Manipulations (Session S1)

```
INSERT INTO transaction_tp VALUES ('T20', 2000);
INSERT INTO transaction_tp VALUES ('T21', 2100);
```

Puis fermer brutalement la session.

✔ Observation

Les données sont perdues après reconnexion.

✔ Conclusion

Une fermeture brutale annule toutes les transactions non validées.

🔹 5. Test DDL + ROLLBACK

```
INSERT INTO transaction_tp VALUES ('T30', 3000);

ALTER TABLE transaction_tp
ADD val2Transaction NUMBER(10);

ROLLBACK;
```

✔ Observation

- La colonne `val2Transaction` reste présente
- L'insertion peut être annulée

✔ Conclusion

Les commandes DDL (`ALTER`, `CREATE`, `DROP`) provoquent un **COMMIT implicite**.

🔹 6. Définitions

🔸 Session

Une session est une connexion entre un utilisateur et le système de gestion de base de données.

🔸 Transaction

Une transaction est un ensemble d'opérations SQL exécutées comme une unité logique de travail.

🔹 7. Validation et annulation

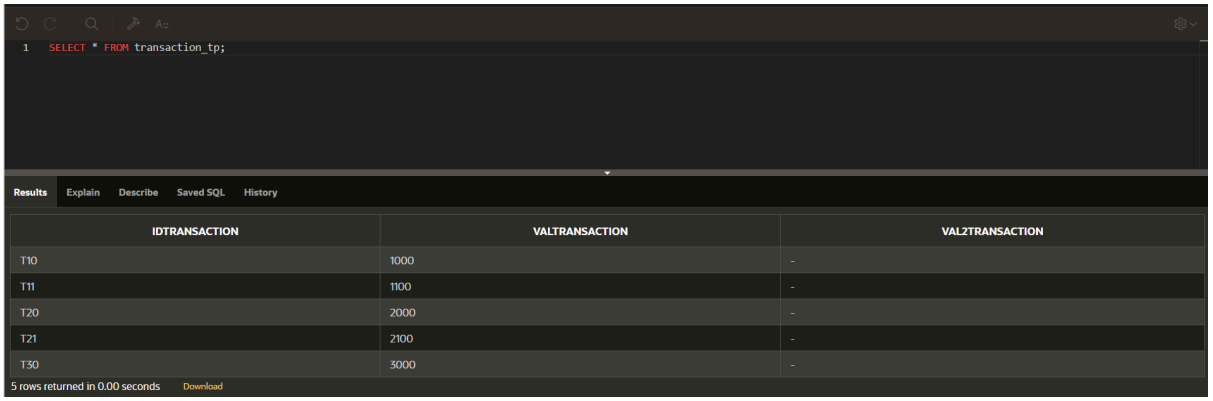
✔ Valider une transaction
COMMIT;

✔ Annuler une transaction
ROLLBACK;

🧠 🎯 Conclusion générale

Ce TP a permis de comprendre le fonctionnement des transactions dans Oracle, en particulier la notion d'atomicité. Les expériences réalisées montrent que les modifications ne deviennent permanentes qu'après un `COMMIT`. En revanche, un `ROLLBACK` ou une fermeture de session annule toutes les opérations non validées. Enfin, les commandes DDL effectuent automatiquement un `COMMIT`, ce qui influence le comportement des transactions.

TEST :



IDTRANSACTION	VALTRANSACTION	VAL2TRANSACTION
T10	1000	-
T11	1100	-
T20	2000	-
T21	2100	-
T30	3000	-

SQL Commands

Language SQL Rows 10 Clear Command Find Tables

```sql
INSERT INTO transaction_tp
VALUES ('TEST1', 999, NULL);
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.01 seconds

→ Apres Commit :

SQL Commands Schema WKSP_BASEDONNEES

Language SQL Rows 10 Clear Command Find Tables Save Run

```sql
SELECT * FROM transaction_tp;
```

Results Explain Describe Saved SQL History

IDTRANSACTION	VALTRANSACTION	VAL2TRANSACTION
T10	1000	-
T11	1100	-
T20	2000	-
T21	2100	-
T30	3000	-
TEST1	999	-

6 rows returned in 0.15 seconds Download